Problem Formulation: Mini Neural Architecture Search (NAS) Controller

Unit 5: Real-World Reinforcement Learning • Project 17 | Target: CIFAR-10 Compact CNNs | Algorithm: REINFORCE with Baseline

1. PROBLEM MOTIVATION & NAS FORMULATION

Manually designing neural network architectures requires tedious empirical trial-and-error and extensive domain intuition. Neural Architecture Search (NAS) automates this pipeline by formulating network design as an optimal sequential decision process. We implement an autoregressive Recurrent Neural Network (RNN) Controller trained via Policy Gradient (REINFORCE with Baseline) that generates discrete topological decisions specifying layer filters, kernel dimensions, activation functions, and residual skip connections.

2. DISCRETE SEARCH SPACE FORMULATION (46,656 TOPOLOGIES)

For each layer $l \in \{1, 2, 3\}$, the controller generates 4 categorical tokens:

- **Filter Channels:** {16, 32, 64} (3 options)
- **Kernel Dimensions:** {3×3, 5×5} (2 options)
- **Activation Functions:** {ReLU, LeakyReLU, GELU} (3 options)
- **Residual Skip Connections:** {None, Residual Add} (2 options)

Total Combinatorial Search Space: $(3 \times 2 \times 3 \times 2)^3 = 36^3 = 46{,}656$ candidate CNN architectures

3. AUTOREGRESSIVE STATE & ACTION SEQUENCES

The RNN controller's hidden state $\mathbf{h}_t$ acts as state. The action sequence $\alpha = (a_1, \ldots, a_{12})$ is generated autoregressively:

$$P(\alpha; \theta) = \prod_{t=1}^{12} P(a_t \mid a_{<t}; \theta)$$

4. REWARD SIGNAL (&R;)

Reward balances validation accuracy against parameter footprint to encourage edge efficiency:

$$R(\alpha) = 100 \times \text{ValAcc}(\alpha) - 0.05 \times [\, \text{Params}(\alpha) / 10{,}000 \,]$$

- ValAcc(α) represents accuracy achieved by candidate network on CIFAR-10 after fast training.
- The parameter penalty discourages bloated configurations and biases search toward compact models.

5. ALGORITHM: REINFORCE WITH EMA BASELINE

Controller parameters θ are updated via Policy Gradient with an Exponential Moving Average baseline:

$$\nabla_\theta J(\theta) = \sum_{t=1}^{12} \nabla_\theta \log P(a_t \mid a_{<t}) \times [\, R(\alpha) - b \,]$$

Baseline update: $b \leftarrow 0.85 \times b + 0.15 \times R$. The advantage term $(R - b)$ ensures low-variance updates.

6. EMPIRICAL SEARCH RESULTS VS. RANDOM SEARCH BASELINE

Search Strategy	Method Type	Best Val Acc	Mean Reward	Top Discovered Architecture
REINFORCE Controller	Autoregressive RNN	**42.13%**	**37.72**	**L0:** 64f (5×5, LeakyReLU, Res) • **L1:** 16f (3×3, LeakyReLU) • **L2:** 32f (5×5, GELU, Res)
Random Architecture Search	Uniform Stochastic	41.13%	37.48	Uncorrelated random configurations across the space